Correa, García, Torterolo

Bases de Datos – Obligatorio

Informe

En el siguiente informe se abordará la fundamentación de las decisiones tomadas a lo largo de la realización del obligatorio, así como un apartado en el que se explicita la división de roles.

Decisiones de implementación:

La primer decisión relevante fue utilizar el framework "Flask" para Python. Esta elección se basó, principalmente, en que es tanto muy flexible como extensible –gracias a extensiones gratuitas e intuitivas para su uso, como lo es Flask-CORS. La búsqueda surgió a partir de la condición del no uso de ORM; se tomó a Flask como la alternativa más adecuada –en comparación a FastAPI–.

Otra decisión de diseño fue la de trabajar el frontend con React Native. Esto se debe a, sencillamente, que es en lo que aprendimos a desarrollar este semestre en el curso de Desarrollo Web y Mobile. Ya que el sistema a desarrollar funciona como una aplicación, más que simplemente como una página web, es que se decantó por romper esa barrera del desarrollo en HTML y CSS "pelado". Dicha herramienta brinda la posibilidad de usarlo desde un celular, un añadido que nos resultó interesante y realista para el caso.

Para la encriptación de contraseñas para iniciar sesión, inicialmente decidimos utilizar la librería *hashlib*, con el algoritmo SHA-256. Luego, al investigar un poco más, la cambiamos por la función *bcrypt*, ya que es muchísimo más segura. En un inicio nos decantamos por una función rápida que no nos convenció al compararlo con la robustez provista por el nuevo método

Se decidió crear un archivo *validaciones.py*, que, justamente, cubre las validaciones necesarias para que cada valor ingresado por el usuario cumpla las reglas deseadas. Junto a este, se creó el archivo *errores.py*, en el que se cubren los errores más comunes que un usuario pueda tener. Algunos ejemplos son: no correr los scripts SQL previamente, no cumplir las restricciones o crear un registro duplicado.

Además, se cargaron todas las librerías necesarias para que el sistema funcione correctamente en *librerias_a_instalar.txt*. El instructivo adjuntado en el README ya lo

cubre, pero nos pareció una idea útil para que, al usuario correr únicamente dicho archivo –y los scripts SQL–, ya se le permita correr el sistema.

Cómo es el flujo de la aplicación:

En primer lugar, es importante aclarar que la app nunca toca la base de datos directamente, sino que le pide lo que se precise al backend. El siguiente diagrama explica muy precisamente el camino de llamadas:

App (React Native) —(HTTP + JSON)—> Backend (Flask) —(SQL)—> MySQL

frontend/ backend/ torneo_futbol5

Consultas extras:

Se agregaron las siguientes tres consultas: una muestra de los jugadores más tarjeteados, una que muestra los goles convertidos por facultad y por último, la edad promedio de los equipos. Se tuvo en cuenta la "utilidad ficticia" de lo que se quería lograr a la hora de desarrollar dichas consultas, es decir, que si fuera un sistema realmente utilizado, serviría tener dichos datos.

Bitácora:

25/10 – Hicimos la planificación del proyecto, analizando qué herramientas nos convenía usar y cómo iba a ser el flujo de la aplicación.

28/10 – Empezamos a diseñar el frontend, ajustando detalles según cómo nos parecía más adecuado. A su vez, probamos cómo conectarnos a una base de datos usando mysql-connector.

29/10 – Cargamos los scripts SQL que contienen las tablas y los permisos para usuarios. Avanzamos en el desarrollo del frontend, y comenzamos a desarrollar el backend (con ayuda de la herramienta de inteligencia artificial "Claude").

30/10 – Cargamos tanto el backend como el frontend. Nos dimos cuenta de que había cosas que no nos gustaban tanto en el diseño, y pensamos en hacerla disponible para mobile también, por lo que nos propusimos migrar el frontend a React Native.

01/11 – Comenzamos la migración a React Native.

03/11 – Cargamos el script con los datos de prueba. Cargamos el nuevo frontend, ahora sí de nuestro agrado. Decidimos simplificar partes del backend que no lográbamos entender al 100% (ayudándonos por Claude). Además, modificamos la función de encriptación de contraseñas al iniciar sesión a *bcrypt*.

04/11 – Diseñamos las consultas SQL correspondientes para cumplir con los requerimientos de la consigna.

Comenzamos a detallar el instructivo de la aplicación, mostrando su flujo y el paso a paso de descargas necesarias para correrla.

05/11 –

División de roles:

Backend – Correa

Frontend – García

Sistema de gestión en MySQL – Todos

Planificación – Todos

Documentación – Torterolo

Bibliografía:

Dolphin, C. (s.f.). Flask-CORS. Read the Docs. https://flask-cors.readthedocs.io/en/latest/

How do I connect SQL server database to a backend? (s.f.). Quora. https://www.quora.com/How-do-I-connect-SQL-server-database-to-a-backend

Pallets. (s.f.). Flask documentation. https://flask.palletsprojects.com/en/stable/

Python Software Foundation. (s.f.). Web frameworks for Python. Python Wiki. https://wiki.python.org/python/web/WebFrameworks.html

Webempresa. (s.f.). Bcrypt: qué es y para qué funciona. https://www.webempresa.com/blog/bcrypt-que-es-y-para-que-funciona.html

OWASP Foundation. (s.f.). Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html